Remise finale du projet (60%)

[Veuillez noter que des ajustements peuvent être apportés à ce document pour l'adapter à la matière qui sera vue durant le reste de la session]

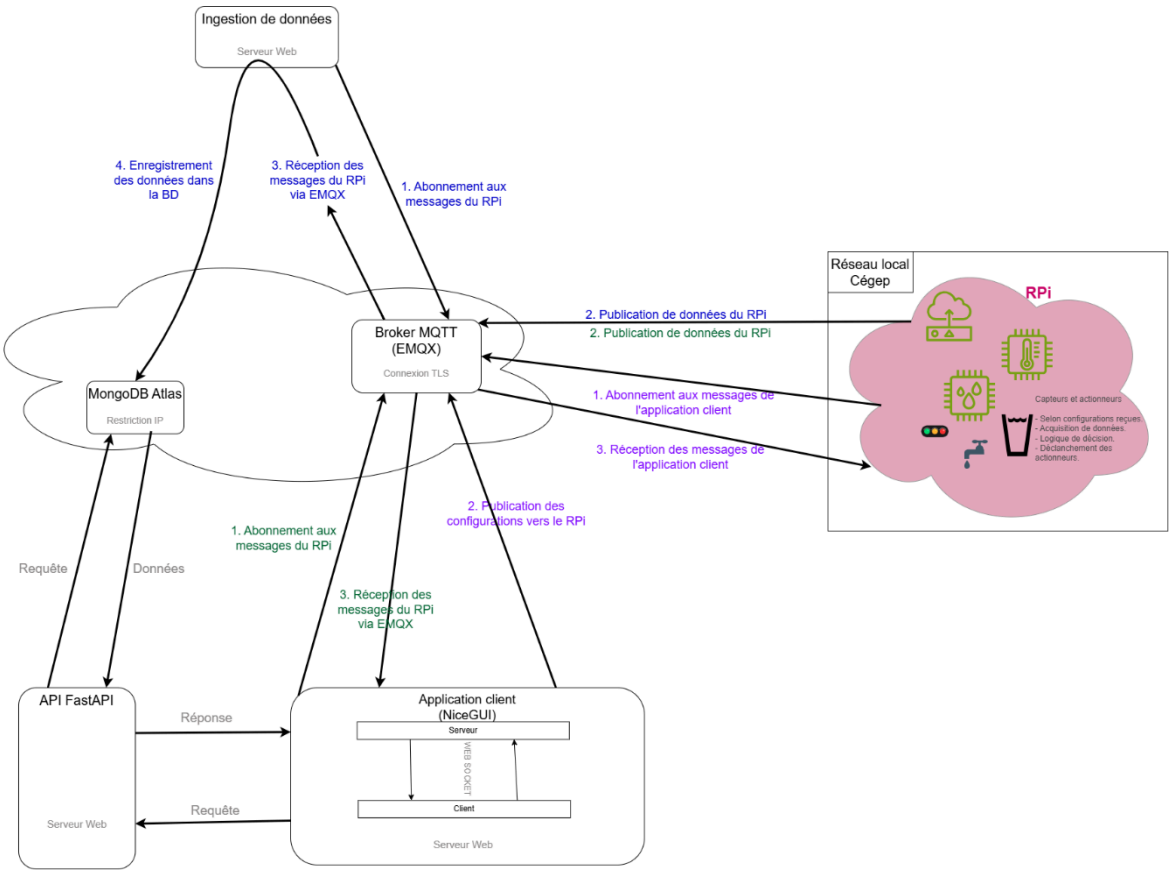
Remise

Dans ce travail, vous devez remettre la version finale de votre projet de session : l'objet connecté, le pipeline de données complet et l'application cliente de contrôle et de surveillance, déployés et fonctionnels.
Votre dépôt GitHub doit inclure :

- le code de l'objet connecté (Raspberry Pi) : acquisition, logique de décision, actionneurs, communication MQTT et configuration à distance;
- le service d'ingestion des données;
- l'API FastAPI;
- l'application cliente NiceGUI;
- les tests d'intégration et d'instrumentation;
- un fichier `requirements.txt` à jour par application et un fichier `.env.example`;
- le README complet, incluant le tableau des tâches.
- **Structure GitHub suggérée** : Créer une organisation avec tous vos projets dedans.

Date de la remise : vendredi, le 11 décembre 2026 à 23h59.

Architecture et déploiement



Exigences fonctionnelles

Votre application doit respecter vos spécifications et maquettes corrigées et à jour.

- **Données réelles** : L'application cliente affiche les données réelles du Raspberry Pi. Commencez à faire votre collecte de données le plus tôt possible et sauvegardez les dans la base de données.
  Laissez vos capteurs et actionneurs fonctionner au moins 48 heures avant la présentation afin d'afficher un historique de données réelles; les données simulées sont permises seulement dans vos tests ou, si votre collecte a été interrompue. Ne mélangez pas vos données simulées, si vous en avez, aux données réelles.
- **Pages de l'application client** : l'application client doit contenir une page d'authentification, un tableau de bord montrant l'état du système (état normal, état d'alerte), des graphiques de suivi des données et activités de l'application, elle doit permettre de faire des configurations et des contrôles manuels.
- **Mise à jour en temps réel et notifications** : l'état de l'application (les valeurs courantes, l'état des actionneurs et l'état du système en général) se met à jour en temps réel. L'utilisateur doit recevoir des notifications de votre choix (une par personne étudiante).
- **Pilotage manuel** : l'utilisateur peut piloter les actionneurs depuis l'application (ex. forcer l'arrosage, activer ou désactiver le mode automatique, etc.).

- **Configuration des capteurs** : l'utilisateur peut modifier la configuration depuis l'application, par exemple la fréquence d'acquisition ou les seuils. Le Raspberry Pi applique la nouvelle configuration et l'application affiche la configuration appliquée.
- **Historiques et graphiques** : L'application doit afficher des graphiques de suivi des données et conserver l'historique des données et des activités (ex. données acquises, alertes, changements d'état des actionneurs, modifications de la configuration). Chaque personne étudiante doit réaliser au moins un graphique, alimenté par l'historique enregistré dans la base de données.
- **Validation des saisies** : les saisies de l'utilisateur doivent toutes être validées, avec des messages clairs en cas d'erreur.
- **Authentification** : les pages de l'application sont protégées par une page de connexion.
- **Notification par courriel** : au moins un événement de l'objet (ex. changement d'état, alerte) déclenche l'envoi d'un courriel (un par étudiant), avec un délai minimal entre deux envois pour éviter les rafales.
- **Dates et heures** : les dates et heures sont affichées à l'heure locale, dans un format québécois (ex. 2026-12-08 14 h 30). Voir l'écriture des dates et heures dans la vitrine linguistique.

Exigences techniques

- **Structure du projet attendue** : Votre projet doit respecter les bonnes pratiques de structuration vues en classe : un environnement virtuel (.venv) et un fichier requirements.txt à jour, listant les librairies utilisées, avoir un fichier d'environnement et un fichier pour les informations sensibles, un fichier de configuration de l'application, etc.

- **Documentation des applications** : Compléter le README de votre première partie du projet selon les mêmes exigences, entre autres :

    - Compléter les dépendances requirements.txt,
    - Compléter les instructions de l'installation des différentes parties de votre système.
    - Compléter les instructions du lancement de la collecte de données.
    - …
- **Broker MQTT** : quand vous serez prêts à déployer, utilisez EMQX (MQTT infonuagique) pour tous les échanges entre le Raspberry Pi, l'ingestion et l'application cliente.
- **Base de données NoSQL MongoDB** : le service d'ingestion enregistre les données reçues du broker dans MongoDB Atlas, selon votre modélisation corrigée.
    - Les horodatages sont enregistrés en UTC dans la base de données.
- **API FastAPI** : l'application cliente obtient l'historique par l'API.
- **Application cliente NiceGUI** : l'application s'abonne au broker MQTT pour les mises à jour en temps réel et les notifications, et publie les commandes et la configuration au Raspberry Pi.
    - *Vous n'avez pas besoin d'utiliser des sockets pour garder la connexion entre le client et le serveur, elles sont déjà intégrées dans NiceGUI.*
- **Validation des données** : sur le Raspberry Pi (valeurs hors plage, configuration invalide), dans l'API et dans l'application cliente.
- **Déploiement** : chaque partie est déployée à l'endroit prévu (voir le tableau d'hébergement) et fonctionne en contexte réel.

Uniquement si le temps le permet et que nous l'aurions vu en classe :

- **Validation JSON (schéma) des données** à l'envoi et à la réception de la base de données.
- **JWT** : si la notion est vue en classe, l'API sera protégée par des jetons JWT.

Coéquipier IA

Votre équipe comprend un coéquipier IA. Donnez-lui un nom et une personnalité de votre choix (soyez créatifs et amusez-vous!). Comme les autres membres, il a des tâches précises. Quelques exemples de tâches que vous pourrez donner à l'IA vont suivre.
Vous restez responsables de tout le code remis :

- chaque personne révise le code produit par l'IA pour sa partie, et doit pouvoir l'expliquer et le modifier pendant la présentation;
- le code produit par l'IA respecte vos exigences, votre modélisation et la structure de votre projet;

les mentions d'auteur (`@Author`) en commentaires indiquent qui a fait quoi, y compris votre coéquipier IA.

Tableau des tâches

Ajoutez dans le README un tableau des tâches de chaque membre de l'équipe, incluant les tâches faites par votre coéquipier IA. Pour chaque tâche confiée à l'IA, donnez un résumé de la demande : ce qui a été fourni (maquette, spécifications, fichiers), ce qui a été demandé et les corrections apportées ensuite.

Tâche	Responsable	Fichiers	Demande faite à l'IA (lorsque ça s'applique)	Corrections apportées

Grille d'évaluation

[… la grille sera mise à jour dans les prochains jours]

Élément évalué	100 %	80 %	60 %	40 %	0 %
1. Analyse du projet, préparation de l'environnement et connexion des objets (00SX.1, 00SX.2)					**25 points**
Identification et branchement des composants **Montage physique** **/ 10 points**	**10 pts** Le montage correspond entièrement aux exigences fonctionnelles ; tous les capteurs et actionneurs prévus sont présents et fonctionnels. Chaque capteur et actionneur est correctement identifié et branché : broches appropriées, protocole respecté, alimentation adéquate.	**8 pts** Le montage correspond aux exigences, avec un écart mineur (ex. composant non essentiel absent ou modifié sans justification). Branchements fonctionnels, avec une erreur mineure de bonne pratique (ex. broche mal choisie, câblage peu soigné).	**6 pts** Le montage correspond partiellement aux exigences ; au moins un composant essentiel, prévu et disponible est absent ou non fonctionnel. Plusieurs erreurs de branchement ou d'identification ; certains composants fonctionnent mal, sont instables (lorsque le défaut ne vient du matériel lui-même).	**4 pts** Le montage ne correspond pas aux exigences fonctionnelles ou est majoritairement non fonctionnel. Branchements majoritairement incorrects ou risqués (mauvaise alimentation, protocole non respecté).	**0 pt** Absent.

Environnement de développement **/ 10 points**	**10 pts** Raspberry Pi connecté au réseau et accessible par SSH; IDE distant configuré (VS Code et Remote-SSH). Interfaces I2C/GPIO activées. Connexion au broker MQTT fonctionnelle Environnement virtuel fonctionnel, requirements.txt présent et à jour, dépendances installées.	**8 pts** Configuration fonctionnelle, avec un élément mineur à corriger (ex. une librairie installée en dehors de l'environnement virtuel). Environnement fonctionnel ; requirements.txt incomplet ou contenant des dépendances superflues.	**6 pts** Plusieurs éléments mal configurés (ex. IDE distant non fonctionnel, I2C/GPIO non activés, connexion au broker impossible). Environnement partiellement fonctionnel ; .venv ou requirements.txt absent, ou installation nécessitant beaucoup de corrections.	**4 pts** Connexion au Raspberry Pi impossible (réseau ou SSH non fonctionnel). Environnement non fonctionnel ; le projet ne peut pas être installé ou lancé.	**0 pt** Absent.
2. Programmation rigoureuse de la logique applicative de l'objet (00SX.4)				**45 points**	
Acquisition des données **/ 10 points**	**10 pts** Fréquence de lecture, unités et seuils cohérents et conformes aux exigences ; intervalle minimal du matériel respecté ; toutes les mesures sont horodatées.	**8 pts** Acquisition conforme, avec un écart mineur (ex. incohérence dans les unités, une mesure non horodatée).	**6 pts** Plusieurs écarts non justifiés aux exigences (fréquence ou intervalle minimal non respecté, horodatage absent pour plusieurs mesures).	**4 pts** Acquisition non fonctionnelle ou sans lien avec les exigences fonctionnelles.	**0 pt** Absent.
Déclenchement des actionneurs **/ 10 points**	**10 pts** La logique respecte toutes les conditions précisées à la planification (seuils, durée).	**8 pts** Logique conforme, avec une condition mal gérée (ex. durée ignorée) ou un cas limite non traité.	**6 pts** Logique partiellement conforme ; seuils incorrects ou plusieurs conditions non respectées.	**4 pts** Aucun actionneur ne se déclenche ou la logique ne correspond pas aux conditions planifiées.	**0 pt** Absent.

Communication MQTT **/ 15 points**	**15 pts** Publication des données, abonnement et écriture dans un fichier fonctionnels, sans perte de données observable.	**12 pts** Chaîne complète fonctionnelle, avec un problème mineur (ex. payload au format incohérent, perte ponctuelle de données).	**9 pts** Chaîne incomplète : la publication fonctionne, mais l'abonnement ou l'écriture dans le fichier est défaillant.	**6 pts** Communication MQTT non fonctionnelle.	**0 pt** Absent.
Structure du code et modélisation orientée objet **/ 10 points**	**10 pts** Séparation claire entre acquisition, logique de décision et communication MQTT; classes, propriétés et méthodes appropriées ; héritage ou classes abstraites lorsque pertinent.	**8 pts** Structure cohérente ; quelques responsabilités mélangées ou une notion OO pertinente non exploitée (ex. code dupliqué au lieu de l'héritage).	**6 pts** Séparation faible des responsabilités ; modélisation OO partielle (classes peu cohésives, logique surtout hors des classes).	**4 pts** Code monolithique (tout dans le même fichier) ; absence de modélisation orientée objet.	**0 pt** Absent.
Gestion des erreurs **/ 5 points**	**5 pts** Les cas d'erreur de base sont gérés (ex. capteur non détecté, valeur hors plage) sans arrêt brutal du programme.	**4 pts** La plupart des cas d'erreur de base sont gérés; un cas trivial n'est pas traité.	**3 pts** Gestion d'erreurs minimale ou trop générique (ex. except global sans traitement).	**2 pts** Aucune gestion d'erreur ; le programme s'arrête au premier problème.	**0 pt** Absent.
3. Documentation, terminologie du domaine et qualité de la langue (00SX.7)					**30 points**
Commentaires du code **/ 10 points**	**10 pts** Toutes les méthodes documentées ; commentaires pertinents sur les constantes, les valeurs et la logique non évidente à la lecture.	**8 pts** Toutes les méthodes sont documentées ; quelques éléments non évidents (exemple : formule, conversion) ne sont pas clarifiés en commentaires.	**6 pts** La majorité des méthodes sont documentées.	**4 pts** Aucune méthode documentée.	**0 pt** Absent.

README **/ 10 points**	**10 pts** README complet : objectif du projet, capteurs et actionneurs, installation (.venv, requirements.txt), lancement de la collecte et emplacement des données.	**8 pts** README complet, mais un élément est imprécis ou incomplet.	**6 pts** README incomplet : plusieurs éléments requis manquants.	**4 pts** README absent ou inutilisable.	**0 pt** Absent.
Terminologie et qualité du français **/ 10 points**	**10 pts** Terminologie du domaine utilisée correctement (broker, topic, payload, seuil, GPIO, I2C); français correct, sans fautes majeures.	**8 pts** Terminologie correcte ; quelques fautes d'orthographe ou de grammaire.	**6 pts** Terminologie parfois imprécise ou erronée ; fautes fréquentes.	**4 pts** Terminologie mal maîtrisée ; fautes nombreuses qui nuisent à la compréhension.	**0 pt** Absent.
					Total : / 100 points